Hashing, Hash Tables and Hash Functions

Data Structures Using C – Lecture 9

Department of Computer Science and Engineering

Graphic Era (Deemed to be University), Dehradun

Recap: Searching Techniques

Technique	Requirement	Time
Linear Search	Unsorted/sorted	$O(n)$
Binary Search	Sorted array	$O(\log n)$
Jump Search	Sorted array	$O(\sqrt{n})$
Fibonacci Search	Sorted array	$O(\log n)$

Important Question

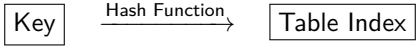
Can we search for an element in

$$\boxed{O(1)}$$

expected time, without scanning or repeatedly dividing the search interval?

Introduction to Hashing

Hashing is a technique used to map a key to a specific position in a table.



- The table is usually implemented using an array.
- The key identifies the record.
- A hash function computes the storage location.
- Hashing supports efficient insertion, searching and deletion.

Motivating Example: Employee Records

Consider a company with 100 employees.
Each employee has an ID:

$$0, 1, 2, \ldots, 99$$

We can store employee records as:

$$\text{Employee}[Emp_ID]$$

For example:

$$Emp_ID = 42$$

The record is directly accessed using:

$$\text{Employee}[42]$$

Observation

The employee ID acts directly as the array index.

Direct Addressing

Employee ID	Array Location
0	0
1	1
2	2
⋮	⋮
42	42
⋮	⋮
99	99

$$\boxed{\text{Index} = \text{Key}}$$

This technique is called **direct addressing**.

Problem with Direct Addressing

Suppose employee IDs now contain five digits.

$$00000 \text{ to } 99999$$

The company still has only:

$$100 \text{ employees}$$

But direct addressing would require:

$$100000 \text{ array locations}$$

Problem

Most array locations would remain unused, resulting in significant memory wastage.

Solution: Convert Keys into Indices

Instead of storing records directly at their full employee ID values, convert each key into a smaller table index.

Example:

$$Emp_ID = 79439$$

Use the last two digits:

$$79439 \bmod 100 = 39$$

Another employee:

$$12345 \bmod 100 = 45$$

Result

The hash function maps large keys to a smaller range of table indices.

What is a Hash Table?

A **hash table** is a data structure in which keys are mapped to array positions using a hash function.

For a key k:

$$\boxed{\text{Index} = h(k)}$$

where:

- k is the key.
- h is the hash function.
- $h(k)$ is the calculated table index.

The process of computing and using these indices is called **hashing**.

Hashing: Basic Illustration

Let:

$$h(k) = k \bmod 10$$

Consider the keys:

$$23, \quad 45, \quad 67, \quad 89$$

Key	Calculation	Index
23	23 mod 10	3
45	45 mod 10	5
67	67 mod 10	7
89	89 mod 10	9

Hash Table After Insertion

For:

$$h(k) = k \bmod 10$$

Index	0	1	2	3	4	5	6	7	8	9
Value	–	–	–	23	–	45	–	67	–	89

To search for 67:

$$h(67) = 67 \bmod 10 = 7$$

Check:

$$T[7] = 67$$

Element Found

Direct Addressing vs Hashing

Property	Direct Addressing	Hashing
Index	Key itself	Computed from key
Table size	Based on key universe	Based on selected capacity
Memory usage	Can be large	Often more compact
Collisions	None for distinct keys	Possible
Lookup	$O(1)$	Expected $O(1)$

Important Terminology

Key: Value used to identify a record.

Hash Function: Function that maps a key to a table index.

Hash Value: Output of the hash function.

Hash Table: Array-based storage structure.

Slot: Individual location in the hash table.

Collision: Two different keys mapping to the same slot.

What is a Hash Function?

A hash function is a mathematical rule that transforms a key into a valid hash table index.

For a table of size m:

$$h : K \to \{0, 1, \ldots, m-1\}$$

Goal

Distribute keys across the available table locations as evenly as possible.

Properties of a Good Hash Function

The textbook identifies three important properties.

1. **Low Cost**
   - Fast to compute.
   - Requires relatively few operations.
2. **Determinism**
   - The same key produces the same hash value for a fixed table configuration.
3. **Uniformity**
   - Keys should be distributed as evenly as possible.

Why is Uniformity Important?

Suppose:

$$m = 10$$

A good hash function attempts to distribute keys across indices:

$$0, 1, 2, \ldots, 9$$

A poor distribution might send many keys to only a few indices.

Consequence

Uneven distribution increases collisions, which can increase insertion and search time.

Different Hash Functions

The textbook discusses four common methods:

1. Division Method
2. Multiplication Method
3. Mid-Square Method
4. Folding Method

We will study the formula, working, numerical example and implementation of each method.

Division Method

The Division Method calculates the remainder when a key is divided by the hash table size.

$$\boxed{h(k) = k \bmod m}$$

where:

- k = key
- m = hash table size
- $h(k)$ = calculated index

For nonnegative keys, the resulting index lies between 0 and $m - 1$.

Division Method: Example 1

Given:

$$m = 10$$

Find hash values for:

$$23, \quad 45, \quad 67, \quad 89$$

$$h(23) = 23 \bmod 10 = 3$$
$$h(45) = 45 \bmod 10 = 5$$
$$h(67) = 67 \bmod 10 = 7$$
$$h(89) = 89 \bmod 10 = 9$$

All four keys map to different locations.

Division Method: Textbook Example

Given:

$$m = 97$$

Calculate:

$$h(1234) = 1234 \bmod 97$$

Since:

$$1234 = 97(12) + 70$$

Therefore:

$$\boxed{h(1234) = 70}$$

For key 5642:

$$5642 = 97(58) + 16$$

$$\boxed{h(5642) = 16}$$

Why Choose a Prime Table Size?

Consider:

$$h(k) = k \bmod m$$

The selection of m affects the distribution of keys.

- Prime values of m are often useful.
- They may reduce patterns caused by common factors.
- Powers of two can make the hash value depend only on low-order bits.

Example

A common table size is:

$$m = 97$$

Division Method: C Function

```c
#include <stdio.h>

#define TABLE_SIZE 97

int hash(int key)
{
    int index = key % TABLE_SIZE;

    if (index < 0)
        index += TABLE_SIZE;

    return index;
}

int main(void)
{
    printf("h(1234) = %d\n", hash(1234));
    printf("h(5642) = %d\n", hash(5642));

    return 0;
}
```

Output

```
h(1234) = 70
```

Multiplication Method

The Multiplication Method uses a constant A satisfying:

$$0 < A < 1$$

The hash function is:

$$\boxed{h(k) = \lfloor m\{kA\} \rfloor}$$

where:

$$\{x\} = x - \lfloor x \rfloor$$

is the fractional part of x.

The textbook discusses the commonly used choice:

$$A \approx 0.6180339887$$

Multiplication Method: Steps

1. Select A such that $0 < A < 1$.
2. Multiply the key k by A.
3. Extract the fractional part of kA.
4. Multiply the fractional part by m.
5. Take the floor of the result.

$$\boxed{k \rightarrow kA \rightarrow \{kA\} \rightarrow m\{kA\} \rightarrow h(k)}$$

Multiplication Method: Textbook Example

Given:

$$k = 12345, \quad m = 1000$$

$$A = 0.618033$$

Step 1:

$$kA = 12345(0.618033) = 7629.617385$$

Step 2:

$$\{kA\} = 0.617385$$

Step 3:

$$m\{kA\} = 617.385$$

Step 4:

$$\boxed{h(12345) = 617}$$

Multiplication Method: C Program

```c
#include <stdio.h>
#include <math.h>

int multiplicationHash(int key, int m)
{
    const double A = 0.618033;

    double product = key * A;
    double fraction = product - floor(product);

    return (int)(m * fraction);
}

int main(void)
{
    int key = 12345;
    int m = 1000;

    printf("Hash value = %d\n",
           multiplicationHash(key, m));

    return 0;
}
```

Compile with: `gcc multiplication.c -o multiplication -lm`

Mid-Square Method

The Mid-Square Method generates a hash value using the middle digits of the square of a key.

1. Square the key.
2. Select a fixed set of middle digits.
3. Use the selected digits as the hash value.

$$\boxed{k \rightarrow k^2 \rightarrow \text{Middle Digits} \rightarrow h(k)}$$

Mid-Square Method: Example 1

Given:

$$k = 1234$$

Square the key:

$$1234^2 = 1522756$$

Suppose the table contains 100 locations.
We need a two-digit index.
Using the textbook's digit-selection convention, select the third and fourth digits counting from the right:

$$15 \underbrace{27}_{\text{Selected}} 56$$

$$\boxed{h(1234) = 27}$$

Mid-Square Method: Example 2

Given:

$$k = 5642$$

Square the key:

$$5642^2 = 31832164$$

Select the third and fourth digits counting from the right:

$$3183 \underbrace{21}_{\text{Selected}} 64$$

Therefore:

$$\boxed{h(5642) = 21}$$

The same digit-selection rule must be used for every key.

Mid-Square Method: Observation

- The key is first transformed by squaring.
- The selected middle digits determine the hash location.
- Different keys may still produce the same hash value.
- The digit-selection convention must remain consistent.

Implementation Note

Squaring very large integer keys may overflow fixed-width integer types.

Mid-Square Method: C Program

```c
#include <stdio.h>

int midSquareHash(int key)
{
    long long square = (long long)key * key;

    /* Select digits 3 and 4 from right */
    return (int)((square / 100) % 100);
}

int main(void)
{
    printf("h(1234) = %d\n",
           midSquareHash(1234));

    printf("h(5642) = %d\n",
           midSquareHash(5642));

    return 0;
}
```

Output

```
h(1234) = 27
h(5642) = 21
```

Folding Method

The Folding Method divides a key into several smaller parts.

1. Divide the key into groups of digits.
2. Add all groups together.
3. Retain the required number of digits for the hash table index.

For a table of 100 locations, groups of two digits can be used.

Folding Method: Example 1

Given:

$$k = 5678, \quad m = 100$$

Divide into groups:

$$56 \quad | \quad 78$$

Add:

$$56 + 78 = 134$$

Retain the last two digits:

$$\boxed{h(5678) = 34}$$

Folding Method: Example 2

Given:

$$k = 321, \quad m = 100$$

Divide:

$$32 \quad | \quad 1$$

Add:

$$32 + 1 = 33$$

Therefore:

$$\boxed{h(321) = 33}$$

Folding Method: Example 3

Given:

$$k = 34567, \quad m = 100$$

Divide:

$$34 \quad | \quad 56 \quad | \quad 7$$

Add:

$$34 + 56 + 7 = 97$$

Therefore:

$$\boxed{h(34567) = 97}$$

Folding Method: Summary

Key	Groups	Sum	Hash Value
5678	$56 + 78$	134	34
321	$32 + 1$	33	33
34567	$34 + 56 + 7$	97	97

For these examples:

$$\boxed{h(k) = \text{Sum of Groups mod } 100}$$

Folding Method: C Function

```c
int foldingHash(int key)
{
    int sum = 0;

    while (key > 0)
    {
        int part = key % 100;
        sum += part;
        key /= 100;
    }

    return sum % 100;
}
```

This function extracts two-digit groups from right to left.
For the textbook examples, it produces the same hash values.

Folding Method: Complete C Program

```c
#include <stdio.h>

int foldingHash(int key)
{
    int sum = 0;

    while (key > 0)
    {
        sum += key % 100;
        key /= 100;
    }

    return sum % 100;
}

int main(void)
{
    printf("h(5678) = %d\n",
           foldingHash(5678));

    printf("h(321) = %d\n",
           foldingHash(321));

    printf("h(34567) = %d\n",
           foldingHash(34567));
```

Comparison of Hash Functions

Method	Main Operation	Example
Division	Remainder	$1234 \bmod 97 = 70$
Multiplication	Fractional part	$h(12345) = 617$
Mid-Square	Square and extract digits	$h(1234) = 27$
Folding	Split and add groups	$h(5678) = 34$

Key Point

No single method guarantees collision-free mapping for every possible set of keys.

What is a Collision?

A collision occurs when two different keys produce the same hash table index.

For example:

$$h(k) = k \bmod 10$$

Consider:

$$k_1 = 23, \quad k_2 = 43$$

Then:

$$h(23) = 3$$

$$h(43) = 3$$

$$\boxed{h(23) = h(43)}$$

Collision: Step-by-Step Example

Suppose:

$$m = 10, \quad h(k) = k \bmod 10$$

Insert key 23:

$$h(23) = 3$$

Index	0	1	2	3	4	5	6	7	8	9
Value	–	–	–	23	–	–	–	–	–	–

Now insert 43:

$$h(43) = 3$$

But:

$$T[3] = 23$$

Collision

Both keys require the same location.

Why Do Collisions Occur?

Suppose:

$$m = 10$$

The table has only ten possible indices:

$$0, 1, \ldots, 9$$

But there may be hundreds or thousands of possible keys.

Pigeonhole Principle

If more than m distinct keys are mapped to m table indices, at least two keys must share an index.

Even fewer than m keys can collide if their hash values are equal.

Collision Resolution Techniques

The textbook introduces two major approaches:

1. **Open Addressing**
   - Linear Probing
   - Quadratic Probing
   - Double Hashing
2. **Chaining**
   - Store multiple keys in a linked list associated with the same index.

Next Lecture

We will implement these techniques and trace their insertion sequences.

What is Load Factor?

The load factor describes how many elements are stored relative to the hash table capacity.

$$\boxed{\alpha = \frac{n}{m}}$$

where:

- n = number of stored elements
- m = number of hash table slots
- α = load factor

Load Factor: Numerical Example

Suppose:

$$m = 10$$

Number of inserted elements:

$$n = 7$$

Then:

$$\alpha = \frac{7}{10} = 0.7$$

$$\boxed{\alpha = 0.7}$$

For an open-addressed table, 70% of its slots are occupied.

Load Factor and Performance

Load Factor	General Interpretation
0.2	Lightly occupied
0.5	Moderately occupied
0.7	Heavily occupied
0.9	Nearly full
1.0	All slots occupied

For open addressing, higher load factors generally increase probing costs.
For chaining, the load factor can exceed 1.

Complexity of Hashing

Operation	Expected	Worst Case
Insertion	$O(1)$	$O(n)$
Search	$O(1)$	$O(n)$
Deletion	$O(1)$	$O(n)$

Important

Expected constant time assumes an effective hash function, controlled load factor and a suitable collision resolution method. It is not a universal worst-case guarantee.

Implementing a Simple Hash Table

We will implement a hash table using:

- An integer array.
- Division Method.
- A marker for empty locations.
- Insertion and search operations.

For now, insertion will reject collisions rather than resolve them.

Purpose

Understand the mapping of keys to indices before introducing probing.

C Program: Hash Table Initialization

```c
#include <stdio.h>

#define SIZE 10

int table[SIZE];
int occupied[SIZE];

void initialize(void)
{
    for (int i = 0; i < SIZE; i++)
    {
        table[i] = 0;
        occupied[i] = 0;
    }
}

int hash(int key)
{
    int index = key % SIZE;

    if (index < 0)
        index += SIZE;

    return index;
}
```

C Program: Insert Operation

```c
int insert(int key)
{
    int index = hash(key);

    if (occupied[index])
    {
        if (table[index] == key)
        {
            printf("Key already present\n");
            return 0;
        }

        printf("Collision at index %d\n",
               index);
        return 0;
    }

    table[index] = key;
    occupied[index] = 1;

    return 1;
}
```

This introductory function detects collisions but does not resolve them.

C Program: Search and Display

```c
int search(int key)
{
    int index = hash(key);

    if (occupied[index] &&
        table[index] == key)
        return index;

    return -1;
}

void display(void)
{
    printf("Index\tValue\n");

    for (int i = 0; i < SIZE; i++)
    {
        if (occupied[i])
            printf("%d\t%d\n", i, table[i]);
        else
            printf("%d\tEMPTY\n", i);
    }
}
```

C Program: Main Function

```c
/* Add this below the previous functions. */

int main(void)
{
    initialize();

    insert(23);
    insert(45);
    insert(67);
    insert(89);

    display();

    int pos = search(67);

    if (pos != -1)
        printf("67 found at index %d\n", pos);
    else
        printf("67 not found\n");

    insert(43); /* Collision with 23 */

    return 0;
}
```

Activity 1: Division Method

Given:

$$m = 11$$

Find the hash values for:

$$25, \quad 36, \quad 47, \quad 58$$

Use:

$$h(k) = k \bmod 11$$

Questions:

1. Calculate each hash value.
2. Identify collisions.
3. Explain why they occur.

Activity 1: Solution

$$h(25) = 25 \bmod 11 = 3$$
$$h(36) = 36 \bmod 11 = 3$$
$$h(47) = 47 \bmod 11 = 3$$
$$h(58) = 58 \bmod 11 = 3$$

All four keys map to:

Index 3

Observation

A hash function may perform poorly for keys with particular numerical patterns.

Activity 2: Mid-Square Method

Given:

$$k = 1234$$

The hash table has:

$$m = 100$$

Tasks:

1. Calculate k^2.
2. Select the third and fourth digits from the right.
3. Determine the hash value.

Activity 2: Mid-Square Method

Given:

$$k = 1234$$

The hash table has:

$$m = 100$$

Tasks:

1. Calculate k^2.
2. Select the third and fourth digits from the right.
3. Determine the hash value.

$$1234^2 = 1522756$$

$$\boxed{h(1234) = 27}$$

Activity 3: Folding Method

Calculate the hash value for:

$$k = 876543$$

Use two-digit groups and a table of 100 locations.

Activity 3: Folding Method

Calculate the hash value for:

$$k = 876543$$

Use two-digit groups and a table of 100 locations.

Groups:

$$87 + 65 + 43$$

Sum:

$$195$$

Hash value:

$$\boxed{h(876543) = 95}$$

Activity 4: Load Factor

A hash table has:

$$m = 50$$

It currently stores:

$$n = 35$$

Calculate its load factor.

Activity 4: Load Factor

A hash table has:

$$m = 50$$

It currently stores:

$$n = 35$$

Calculate its load factor.

$$\alpha = \frac{35}{50}$$

$$\boxed{\alpha = 0.7}$$

MCQ 1

Which data structure maps keys to array indices using a function?

1. Stack
2. Queue
3. Hash Table
4. Binary Tree

MCQ 1

Which data structure maps keys to array indices using a function?

1. Stack
2. Queue
3. Hash Table
4. Binary Tree

Answer

Option 3: Hash Table.

MCQ 2

For:

$$h(k) = k \bmod 10$$

What is:

$$h(87)?$$

1. 8
2. 7
3. 87
4. 10

MCQ 2

For:

$$h(k) = k \bmod 10$$

What is:

$$h(87)?$$

1. 8
2. 7
3. 87
4. 10

Answer

Option 2: 7.

MCQ 3

A collision occurs when:

1. Two keys are equal.
2. Two distinct keys map to the same hash table index.
3. The array is sorted.
4. The table contains no elements.

MCQ 3

A collision occurs when:

1. Two keys are equal.
2. Two distinct keys map to the same hash table index.
3. The array is sorted.
4. The table contains no elements.

Answer

Option 2.

MCQ 4

The load factor is calculated as:

1. m/n
2. $n + m$
3. n/m
4. $n - m$

MCQ 4

The load factor is calculated as:

1. m/n
2. $n + m$
3. n/m
4. $n - m$

Answer

Option 3:

$$\alpha = \frac{n}{m}$$

MCQ 5

Which method squares the key before extracting selected digits?

1. Division Method
2. Folding Method
3. Mid-Square Method
4. Multiplication Method

MCQ 5

Which method squares the key before extracting selected digits?

1. Division Method
2. Folding Method
3. Mid-Square Method
4. Multiplication Method

Answer

Option 3: Mid-Square Method.

Short Answer Questions

1. Define hashing and hash table.
2. Explain direct addressing.
3. Why is hashing preferred over direct addressing for sparse key sets?
4. List the properties of a good hash function.
5. Explain the Division Method.
6. Explain the Multiplication Method.
7. Describe the Mid-Square Method.
8. Explain Folding with an example.
9. Define collision and load factor.
10. Why is collision resolution necessary?

Programming Assignments

1. Implement the Division Method in C.
2. Write a C program for the Multiplication Method.
3. Implement the Mid-Square Method.
4. Implement Folding for two-digit groups.
5. Build a hash table that detects collisions.
6. Modify the hash table program to accept user input.
7. Calculate and display the load factor after every successful insertion.

Lecture Summary

- Hashing maps keys to table indices.
- A hash function computes the location.
- Direct addressing may waste memory for large key universes.
- A good hash function should be fast, deterministic and reasonably uniform.
- Four methods were discussed: Division, Multiplication, Mid-Square and Folding.
- Different keys may collide.
- Load factor describes table occupancy.
- Expected hash table lookup can be $O(1)$.

Collision Resolution Techniques

1. Linear Probing
2. Quadratic Probing
3. Double Hashing
4. Rehashing
5. Chaining
6. Hash Table Implementation in C

Collision $\rightarrow$ Resolution

References

1. Reema Thareja, *Data Structures Using C*, Chapter 15: Hashing and Collision.
2. Thomas H. Cormen et al., *Introduction to Algorithms*, Hash Tables.
3. Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*.

Lecture Feedback

Your Feedback Matters

Please share feedback on:

- Hashing concepts
- Hash functions
- Numerical examples
- C programs
- Collisions and load factor



Scan to Submit Feedback

Thank You

Questions and Discussion

Hashing and Hash Functions